

Atividade Final - Parte Final

Sistema de Gestão Financeira Pessoal e Compartilhada , React + Spring Boot

Etapas finais do projeto. A partir daqui, a atividade não tem nível de detalhamento: você já projetou telas (Parte 1) e uma API (Parte 2). Agora precisa integrar tudo, amadurecer as funcionalidades que fizerem mais sentido para o seu sistema e dar a ele a sua identidade.

Os **requisitos mínimos** abaixo são inegociáveis e serão cobrados de todos. Tudo o que não está definido aqui , nomes, fluxos extras, layout, escopo das funcionalidades novas , é decisão sua. Um bom projeto final não se limita ao mínimo: ele mostra domínio técnico e personalidade.

1. Requisitos Mínimos Obrigatórios

Estes são os pontos que todo projeto precisa atender, independentemente das escolhas de escopo e identidade de cada aluno.

1.1 Integração completa

- Frontend e backend integrados, nada de dados mockados na entrega final.
- Fluxos essenciais (autenticação, carteiras, transações, compartilhamento) funcionando com dados reais persistidos em banco.

1.2 Comunicação em tempo real

- Uso de WebSocket e/ou Server-Sent Events em pelo menos um ponto real do sistema , não apenas um "hello world" de exemplo.
- Exemplos possíveis (escolha livre): saldo/dashboard atualizando ao vivo quando outro membro lança uma transação, notificações em tempo real, chat, indicador de atividade na carteira compartilhada. Você pode propor outro uso, desde que faça sentido para o sistema.

1.3 Relatórios

- Geração de pelo menos um relatório, extrato de transações, resumo mensal, relatório por categoria, etc.

1.4 Integração com IA

- Uso de uma API de IA integrada a uma funcionalidade real do sistema , não decorativa. Exemplos: categorização automática de transações, assistente/chat financeiro, geração de insights ou resumos em linguagem natural sobre os gastos do usuário.
- A escolha do provedor de IA é livre. Documente no README como configurar a chave/API utilizada.
- Sugestão: gemini plano gratuito.

1.5 PWA (Progressive Web App)

- Aplicação instalável (manifest.json configurado).
- Service worker com pelo menos um nível de funcionamento offline (ex.: cache de assets estáticos e/ou última tela visitada).

1.6 Segurança (baseado no OWASP)

- Aplicação de práticas alinhadas ao OWASP Top 10, com foco no que é relevante para o seu sistema: proteção contra injeção (SQL Injection), XSS, CSRF onde aplicável, validação e sanitização de entradas, controle de acesso consistente (broken access control), rate limiting em rotas sensíveis (ex.: login), cabeçalhos de segurança HTTP, e gerenciamento seguro de segredos (sem credenciais hardcoded no repositório).
- Não é necessário cobrir toda a lista do OWASP, identifique os riscos mais relevantes para o seu sistema.

1.7 Arquitetura, qualidade e testes

- Uso de classes/módulos adicionais que demonstrem domínio de orientação a objetos e organização de código (ex.: padrões de projeto camadas de serviço bem definidas, DTOs, etc.).
- Apenas sugestão, uso de testes automatizados (unitários e/ou de integração), no frontend e/ou no backend.

1.8 Implantação (deploy/hospedagem)

- Sistema implantado em ambiente acessível publicamente por um link (ex.: frontend em Vercel/Netlify, backend em Render/Railway/Fly.io/AWS/Azure/Supabase/outro).
- O link de acesso funcional deve estar no README.

1.9 Documentação

- README atualizado: descrição do sistema, arquitetura, como rodar localmente, variáveis de ambiente, link do deploy e principais decisões de projeto.
- Documentação da API (Swagger/OpenAPI) atualizada e acessível.

2. Liberdade e Personalidade

Fora dos requisitos mínimos, o restante do sistema é seu. Você pode adicionar novas funcionalidades, remodelar telas, criar módulos que não foram propostos em nenhuma parte anterior. Quanto mais o projeto refletir uma identidade própria, e não apenas o mínimo pedido, melhor.

Algumas ideias (nenhuma obrigatória) para quem quiser ir além:

- Metas financeiras e acompanhamento de progresso
- Modo escuro / temas personalizáveis
- Notificações push
- Internacionalização (múltiplos idiomas)
- Containerização com Docker / docker-compose
- Pipeline de CI/CD
- Cache (ex.: Redis) para dados de dashboard

- Logs estruturados

3. Apresentação Final

Mostrar o sistema funcionando já implantado e explicar as decisões técnicas tomadas.

4. Critérios de Avaliação

Item
Funcionalidade: sistema integrado, funcionando de ponta a ponta (frontend + backend)
Requisitos mínimos: tempo real, relatórios, IA, PWA e segurança implementados
Qualidade técnica: arquitetura, organização em camadas, testes automatizados
Implantação: sistema acessível publicamente e funcionando no ambiente de deploy
Documentação: README, variáveis de ambiente e Swagger/OpenAPI atualizados
Apresentação/defesa: clareza ao explicar decisões técnicas e personalidade do projeto

5. Prazo de Entrega

Até **19/11/2026**.